

1.Perform basic image handling and processing operations on an image. Read an image using Python and convert the image into Gray-scale.

```
# Import required libraries

import cv2

import matplotlib.pyplot as plt


# Read image

image = cv2.imread("/images.jpg")


# Convert BGR image to Gray scale

gray_image = cv2.cvtColor(
    image,
    cv2.COLOR_BGR2GRAY
)


# Display images

plt.figure(figsize=(8, 4))


# Original image

plt.subplot(1, 2, 1)

plt.imshow(
    cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
)

plt.title("Original Image")

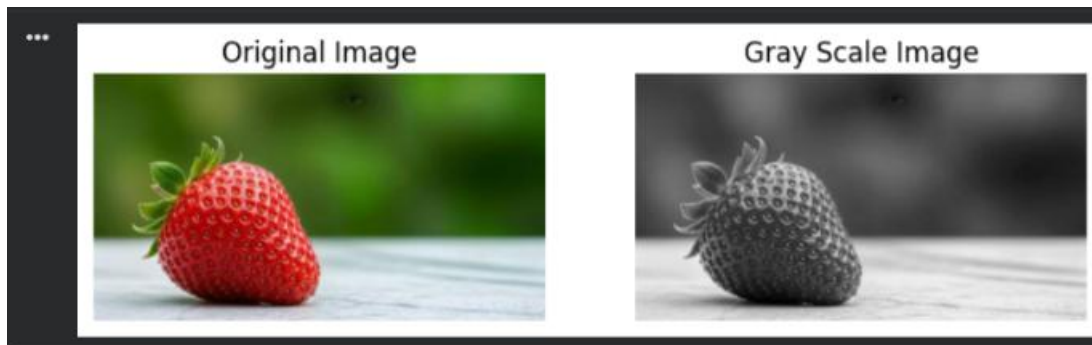
plt.axis("off")


# Gray scale image

plt.subplot(1, 2, 2)

plt.imshow(
    gray_image,
    cmap="gray"
```

```
)  
plt.title("Gray Scale Image")  
plt.axis("off")  
  
plt.show()
```



2. Perform basic image handling and processing operations on an image. Read an image using Python and convert the image into Blur using Gaussian Blur.

```
# Import required libraries  
import cv2  
import matplotlib.pyplot as plt  
  
# Read input image  
image = cv2.imread("/images.jpg")  
  
# Apply Gaussian Blur  
blur_image = cv2.GaussianBlur(  
    image,  
    (7, 7),  
    0  
)  
  
# Display images  
plt.figure(figsize=(8, 4))
```

```
# Original Image
```

```
plt.subplot(1, 2, 1)
```

```
plt.imshow(
```

```
    cv2.cvtColor(
```

```
        image,
```

```
        cv2.COLOR_BGR2RGB
```

```
    )
```

```
)
```

```
plt.title("Original Image")
```

```
plt.axis("off")
```

```
# Gaussian Blur Image
```

```
plt.subplot(1, 2, 2)
```

```
plt.imshow(
```

```
    cv2.cvtColor(
```

```
        blur_image,
```

```
        cv2.COLOR_BGR2RGB
```

```
    )
```

```
)
```

```
plt.title("Gaussian Blur Image")
```

```
plt.axis("off")
```

```
plt.show()
```



3. Perform basic image handling and processing operations on an image. Read an image using Python and display image outline using Canny function.

```
# Import required libraries
```

```
import cv2
```

```
import matplotlib.pyplot as plt
```

```
# Read input image
```

```
image = cv2.imread("/images.jpg")
```

```
# Convert image into grayscale
```

```
gray_image = cv2.cvtColor(
```

```
    image,
```

```
    cv2.COLOR_BGR2GRAY
```

```
)
```

```
# Apply Gaussian Blur
```

```
blur_image = cv2.GaussianBlur(
```

```
    gray_image,
```

```
    (5, 5),
```

```
    0
```

```
)
```

```
# Apply Canny Edge Detection
```

```
edges = cv2.Canny(
```

```
    blur_image,
```

```
100,  
200  
)  
  
# Display images  
plt.figure(figsize=(10, 4))  
  
# Original Image  
plt.subplot(1, 2, 1)  
  
plt.imshow(  
    cv2.cvtColor(  
        image,  
        cv2.COLOR_BGR2RGB  
    )  
)  
  
plt.title("Original Image")  
plt.axis("off")  
  
# Edge Image  
plt.subplot(1, 2, 2)  
  
plt.imshow(  
    edges,  
    cmap="gray"  
)  
  
plt.title("Canny Edge Image")  
plt.axis("off")
```

```
plt.show()
```



4. Implement Histogram Equalization on the given image and compare it with the original image using OpenCV.

```
# Import required libraries
```

```
import cv2
```

```
import matplotlib.pyplot as plt
```

```
# Read image
```

```
image = cv2.imread("image.jpg")
```

```
# Check if image is loaded successfully
```

```
if image is None:
```

```
    print("Error: Could not load image.jpg")
```

```
else:
```

```
    # Convert image to grayscale
```

```
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
# Apply Histogram Equalization
```

```
equalized_image = cv2.equalizeHist(gray_image)
```

```
# Display Original and Equalized Images
```

```
plt.figure(figsize=(10, 4))
```

```
# Original Gray Image
```

```
plt.subplot(1, 2, 1)
plt.imshow(gray_image, cmap="gray")
plt.title("Original Image")
plt.axis("off")
```

Enhanced Image

```
plt.subplot(1, 2, 2)
plt.imshow(equalized_image, cmap="gray")
plt.title("Equalized Image")
plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```

Display Histograms

```
plt.figure(figsize=(10, 4))
```

Original Histogram

```
plt.subplot(1, 2, 1)
plt.hist(gray_image.ravel(), 256, [0, 256])
plt.title("Original Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
```

Equalized Histogram

```
plt.subplot(1, 2, 2)
plt.hist(equalized_image.ravel(), 256, [0, 256])
plt.title("Equalized Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
```

```
plt.tight_layout()
```

```
plt.show()
```



EXPERIMENT 5 Histogram Analysis of Image Based on Color Levels using OpenCV

```
# Import required libraries
```

```
import cv2
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
# Read input image
```

```
image = cv2.imread("/images.jpg")
```

```
# Convert image into grayscale
```

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
# Apply Sobel X direction
```

```
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
```

```
# Apply Sobel Y direction
```

```
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
```



```
# Convert Sobel results to uint8
sobel_x = np.uint8(np.absolute(sobel_x))
sobel_y = np.uint8(np.absolute(sobel_y))

# Combine Sobel X and Sobel Y
sobel_combined = cv2.addWeighted(
    sobel_x, 0.5,
    sobel_y, 0.5,
    0
)

# Display result
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(gray, cmap="gray")
plt.title("Gray Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(sobel_x, cmap="gray")
plt.title("Sobel X")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(sobel_combined, cmap="gray")
plt.title("Sobel Edge Detection")
plt.axis("off")

plt.show()
```

...

Gray Image



Sobel X



Sobel Edge Detection

